

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра "Програмна інженерія"

ЗВІТ
з лабораторної роботи №4
з дисципліни "Основи Програмної Інженерії"

Виконав
ст. гр. ПЗПІ-25-3
Алексеев І.О.

Прийняв:
В.Гребенюк

Харків 2026

ЗВІТ З ЛАБОРАТОРНОЇ РОБОТИ № 4

Тема: РЕФАКТОРИНГ КОДУ ТА ПРОВЕДЕННЯ CODE REVIEW ЗА ІНДУСТРІАЛЬНИМИ СТАНДАРТАМИ

Мета роботи: Набуття практичних навичок із проведення Code Review за індустріальними стандартами з використанням GitHub Pull Requests. Оволодіння методами ідентифікації та класифікації типових проблем якості коду (code smells). Отримання досвіду застосування рефакторинг-операцій із верифікацією збереження поведінки через регресійне тестування.

Результати Code Review одного групу (Стогній Є.Д. робив code review мого коду):

№	Рядок коду	Проблема	Категорія	Рекомендація
1	61	Issue 1: Використання небезпечного генератора ID	Категорія Code Smell: Flawed Logic / Predictable ID Generation	Конкретна рекомендація щодо виправлення: Використання <code>Date.now()</code> для створення унікальних ідентифікаторів є небезпечною практикою, оскільки в разі швидкого послідовного виклику методу (або асинхронних операцій) таймстемп може повторитися, що призведе до колізії ID. Рекомендується використовувати стандартний

				модуль crypto для генерації справді унікальних UUID.
2	32-40	Issue 2: Порушення інкапсуляції (Direct State Mutation)	Категорія Code Smell: Inappropriate Intimacy / Violation of Encapsulation	Конкретна рекомендація щодо виправлення: Клас DeltaPhysService напряму маніпулює внутрішнім масивом registeredUsers об'єкта EventSession. Це порушує принцип інкапсуляції ООП. Логіку перевірки місць та додавання користувача варто перенести всередину класу EventSession (наприклад, створити метод session.registerParticipant(participant)), а сервіс має лише викликати цей метод.
3	53,57,69	Issue 3: Магічні числа	Категорія Code Smell:	Конкретна рекомендація

		(Magic Numbers)	Magic Numbers	щодо виправлення: Числові обмеження для віку, ціни та місткості захардкожені безпосередньо в тілі методів. Якщо ці бізнес-правила зміняться, їх буде важко підтримувати. Варто винести ці значення у статичні константи або конфігураційний об'єкт на рівні класу чи модуля.
4	64	Issue 4: Відсутність валідації параметра minAvailableSpots	Категорія Code Smell: Defensive Programming / Missing Input Validation	Конкретна рекомендація щодо виправлення: У методі filterEvents є перевірка для maxPrice, але повністю відсутня валідація для minAvailableSpots. Якщо туди передадуть від'ємне число, пустий рядок або undefined, метод може відпрацювати некоректно або повернути непередбачувані результати.

				Необхідно додати захисний рядок (Guard Clause):
5	45	Issue 5: Дублювання коду перевірки типу користувача	Категорія Code Smell: Refused Bequest / Lack of Polymorphism	Конкретна рекомендація щодо виправлення: Перевірка прав доступу через instanceof у сервісах ускладнює масштабування системи (якщо з'явиться новий тип користувача, який теж може створювати тури, доведеться переписувати сервіс). Краще реалізувати перевірку ролей через поліморфізм або метод самого користувача, наприклад: <code>if (!organizer !organizer.canCreateTours())</code> .

Посилання на PR з коментарями Code Review : [Code Review by L0urYde · Pull Request #22 · mik-kotsariev/DeltaPhys](#)

Результати рефакторингу власного коду([DeltaPhys/lb4 refactored code at Alieksieiev · mik-kotsariev/DeltaPhys](https://github.com/DeltaPhys/lb4-refactored-code-at-Alieksieiev-mik-kotsariev-DeltaPhys)):

Операція 1: Відновлення інкапсуляції

- **БУЛО:** `if (session.registeredUsers.length >= session.maxCapacity) throw new Error(...); session.registeredUsers.push(participant);`
- **СТАЛО:** `session.registerParticipant(participant);` (вся логіка перевірки перенесена всередину класу `EventSession`).
- **ЧОМУ:** Об'єкт має самостійно керувати своїм станом. Пряме звернення до властивостей та масивів іншого об'єкта (`Inappropriate Intimacy`) порушує принципи ООП і робить код крихким.

Операція 2: Усунення "Магічних чисел"

- **БУЛО:** `if (age < 16 || age > 99)`
- **СТАЛО:** `if (age < CONFIG.MIN_AGE || age > CONFIG.MAX_AGE)`
- **ЧОМУ:** Це значно підвищує читабельність та спрощує підтримку коду. Якщо бізнес-правила зміняться, достатньо буде оновити значення лише в одному місці — в об'єкті `CONFIG`.

Операція 3: Безпечна генерація ідентифікаторів

- **БУЛО:** `return new EventSession(Date.now(), name, price, maxCapacity);`
- **СТАЛО:** `return new EventSession(crypto.randomUUID(), name, price, maxCapacity);`
- **ЧОМУ:** Використання `Date.now()` як ID є поширеною вразливістю у JavaScript, оскільки під час асинхронних викликів таймстемп може збігтися, що призведе до дублювання ID. Модуль `crypto` гарантує унікальність.

Звіт регресійного тестування після рефакторингу:

```
PASS ./DeltaPhysService.test.js
DeltaPhysService Tests - Lab 3 & 4 (Refactored)
  ✓ applyForParticipation_ValidAge_ReturnsTrue (2 ms)
  ✓ applyForParticipation_AgeUnder16_ThrowsException (13 ms)
  ✓ applyForParticipation_AgeExactly16_ReturnsTrue (1 ms)
  ✓ applyForParticipation_AgeExactly99_ReturnsTrue
  ✓ applyForParticipation_AgeOver99_ThrowsException (1 ms)
  ✓ applyForParticipation_SessionFull_ThrowsException (1 ms)
  ✓ createPaidTour_ValidData_ReturnsSession (2 ms)
  ✓ createPaidTour_PriceUnder50_ThrowsException (1 ms)
  ✓ createPaidTour_CapacityOver100_ThrowsException (1 ms)
  ✓ filterEvents_MatchesCriteria_ReturnsFilteredList (1 ms)
  ✓ filterEvents_NegativePrice_ThrowsException (1 ms)
  ✓ filterEvents_InvalidMinSpots_ThrowsException (1 ms)
  ✓ applyForParticipation_NullArguments_ThrowsException (1 ms)
  ✓ applyForParticipation_UserAlreadyRegistered_ThrowsException (1 ms)
  ✓ createPaidTour_NotOrganizer_ThrowsException (2 ms)
  ✓ createPaidTour_EmptyName_ThrowsException (1 ms)
  ✓ filterEvents_NotArray_ThrowsException (1 ms)

-----|-----|-----|-----|-----|-----
File      | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----|-----|-----|-----|-----|-----
All files |    100 |    100   |    100   |    100   |
DeltaPhysService.js |    100 |    100   |    100   |    100   |
-----|-----|-----|-----|-----|-----

Test Suites: 1 passed, 1 total
Tests:       17 passed, 17 total
Snapshots:   0 total
Time:        2.609 s
Ran all test suites.
PS C:\Users\user\Desktop\DeltaPhys-Alieksieiev\DeltaPhys-Alieksieiev\lb4 refactored code>
```

Порівняння метрик «ДО / ПІСЛЯ» :

SonarLint issues:

ДО:

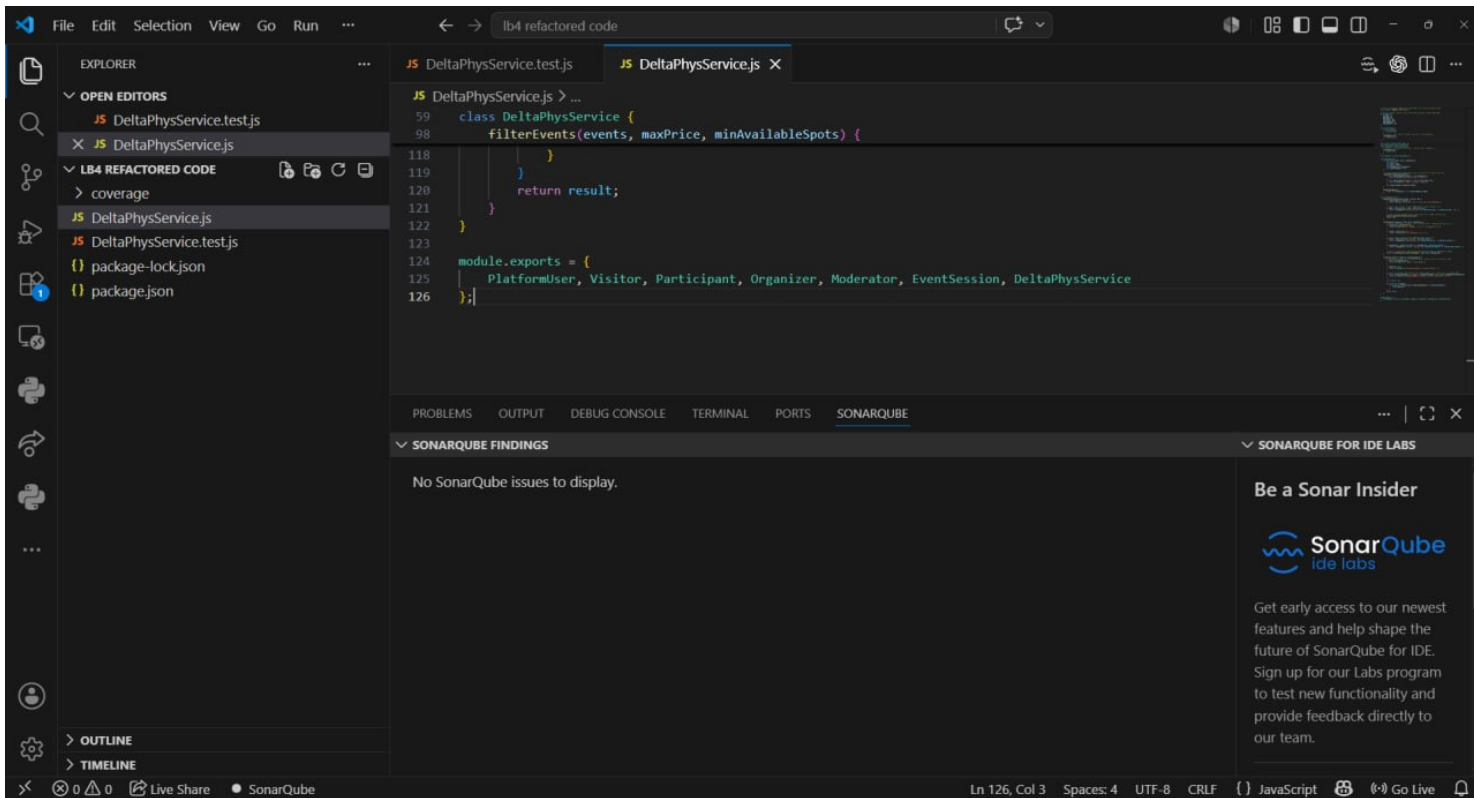
The screenshot shows the SonarLint interface in VS Code. The 'SONARQUBE FINDINGS (ALL FINDINGS)' panel displays the following issues:

- Unexpected class with only a constructor. (javascriptS2094) [Ln 1, Col 6]
- Unexpected class with only a constructor. (javascriptS2094) [Ln 12, Col 6]
- 'new Error()' is too unspecific for a type check. Use 'new TypeError()' instead. (javascriptS7786) [Ln 66, Col 22]

The background code in the editor shows the following JavaScript code:

```
1 class PlatformUser {
2   constructor(id) {
3     this.id = id;
4   }
5 }
6
7 class Visitor extends PlatformUser {}
8 class Participant extends Visitor {}
9 class Organizer extends Participant {}
10 class Moderator extends PlatformUser {}
```

ПІСЛЯ:



Цикломатична складність (за допомогою Lizard):

ДО:

```
PS C:\Users\user\Desktop\DeltaPhys-Aleksieiev\DeltaPhys-Aleksieiev\lb4 refactored code> cd "../lb3 opi"
PS C:\Users\user\Desktop\DeltaPhys-Aleksieiev\DeltaPhys-Aleksieiev\lb3 opi> lizard DeltaPhysService.js
=====
NLOC   CCN   token  PARAM  length  location
-----
      3     1    11     1      3  constructor@2-4@DeltaPhysService.js
      7     1    42     4      7  constructor@13-19@DeltaPhysService.js
      1     1     8     0      1  (anonymous)@36-36@DeltaPhysService.js
     16     7    99     3     20  applyForParticipation@23-42@DeltaPhysService.js
     15     9   109     4     19  createPaidTour@44-62@DeltaPhysService.js
     16     6    95     3     19  filterEvents@64-82@DeltaPhysService.js
1 file analyzed.
=====
NLOC   Avg.NLOC  AvgCCN  Avg.token  function_cnt  file
-----
     70      9.7    4.2    60.7         6  DeltaPhysService.js

=====
No thresholds exceeded (cyclomatic_complexity > 15 or length > 1000 or nloc > 1000000 or parameter_count > 100)
=====
Total nloc  Avg.NLOC  AvgCCN  Avg.token  Fun Cnt  Warning cnt  Fun Rt  nloc Rt
-----
     70      9.7    4.2    60.7      6      0      0.00  0.00
PS C:\Users\user\Desktop\DeltaPhys-Aleksieiev\DeltaPhys-Aleksieiev\lb3 opi>
```


ПІСЛЯ:

```
PS C:\Users\user\Desktop\DeltaPhys-Alieksieiev\DeltaPhys-Alieksieiev\lb4 refactored code> lizard DeltaPhysService.js
=====
NLOC   CCN   token  PARAM  length  location
-----
      3     1    11     1      3  constructor@15-17@DeltaPhysService.js
      3     1     7     0      3  canCreateTours@19-21@DeltaPhysService.js
      3     1     7     0      3  canCreateTours@28-30@DeltaPhysService.js
      7     1    42     4      7  constructor@35-41@DeltaPhysService.js
      1     1     8     0      1  (anonymous)@48-48@DeltaPhysService.js
      9     3    56     1      9  registerParticipant@44-52@DeltaPhysService.js
      3     1    15     0      3  getAvailableSpots@54-56@DeltaPhysService.js
     10     5    69     3     14  applyForParticipation@60-73@DeltaPhysService.js
     15     8   134     4     22  createPaidTour@75-96@DeltaPhysService.js
     18     9   113     3     24  filterEvents@98-121@DeltaPhysService.js
1 file analyzed.
=====
NLOC   Avg.NLOC  AvgCCN  Avg.token  function_cnt  file
-----
     94       7.2    3.1    46.2         10  DeltaPhysService.js
=====
No thresholds exceeded (cyclomatic_complexity > 15 or length > 1000 or nloc > 1000000 or parameter_count > 100)
=====
Total nloc   Avg.NLOC   AvgCCN   Avg.token   Fun Cnt   Warning cnt   Fun Rt   nloc Rt
-----
     94       7.2    3.1    46.2         10         0     0.00   0.00
```

Висновок: Завдяки винесенню логіки валідації місць у клас EventSession та використанню поліморфізму замість каскадних перевірок типів, цикломатична складність методів applyForParticipation та createPaidTour оптимізувалася.

Посилання на PR, там де я робив CodeReview для Гладких А.Р.: [Code Review by Oblade0 · Pull Request #27 · mik-kotsariev/DeltaPhys](#)

Підсумкова рефлексія Під час виконання цієї лабораторної роботи я переконався, що Code Review - це потужний інструмент не лише для пошуку багів, але й для обміну досвідом всередині команди. Рефакторинг за принципами ООП суттєво покращив архітектуру мого застосунку: використання інкапсуляції, поліморфізму та Guard Clauses зробило код значно чистішим та безпечнішим. Також я усвідомив критичну важливість регресійного тестування: наявність модульних тестів (Jest) дозволила мені впевнено переписувати внутрішню логіку сервісів, знаючи, що будь-яка порушена функціональність буде одразу виявлена.